Jonah King

CSCI 499
Senior Project Implementation/Defense
3-D Space Shooter

April 15th, 2019

Bachelor of Science in Computer Science

Table of Contents

Introduction

I decided to make a space shooter for my senior project because I wanted to truly challenge myself and get a better understanding of the game development world. This was also an excellent way to widen my horizon and make me a better overall programmer by giving me experience in a language I haven't seen and made me use resources that I had not previously used before. This project has overall turned me into a more proficient programmer and I am very excited to see where the new skills I have developed will take me in the future. For the game itself, the objective is to play almost perfectly as a death will cause the player to restart the game. I decided to make the player have a perfect playstyle, because I like the idea of a game that demands perfection. If the player can make it all the way to the end, then they will be rewarded with a boss fight and if they can manage to defeat the boss while dodging all the other enemies then the player will be rewarded with 2000 points. Although the Senior Project was the main reason I decided to make a game another reason was for my enjoyment and love for video games and now that I have made one I now consider myself somewhat of a mini game developer. This also gave me a huge understanding of what the bigger companies go through whenever they are releasing a new game and I can say that I will never complain about waiting for a game to come out because I can only imagine what those companies go through as I only went through a small portion of difficulties compared to them.

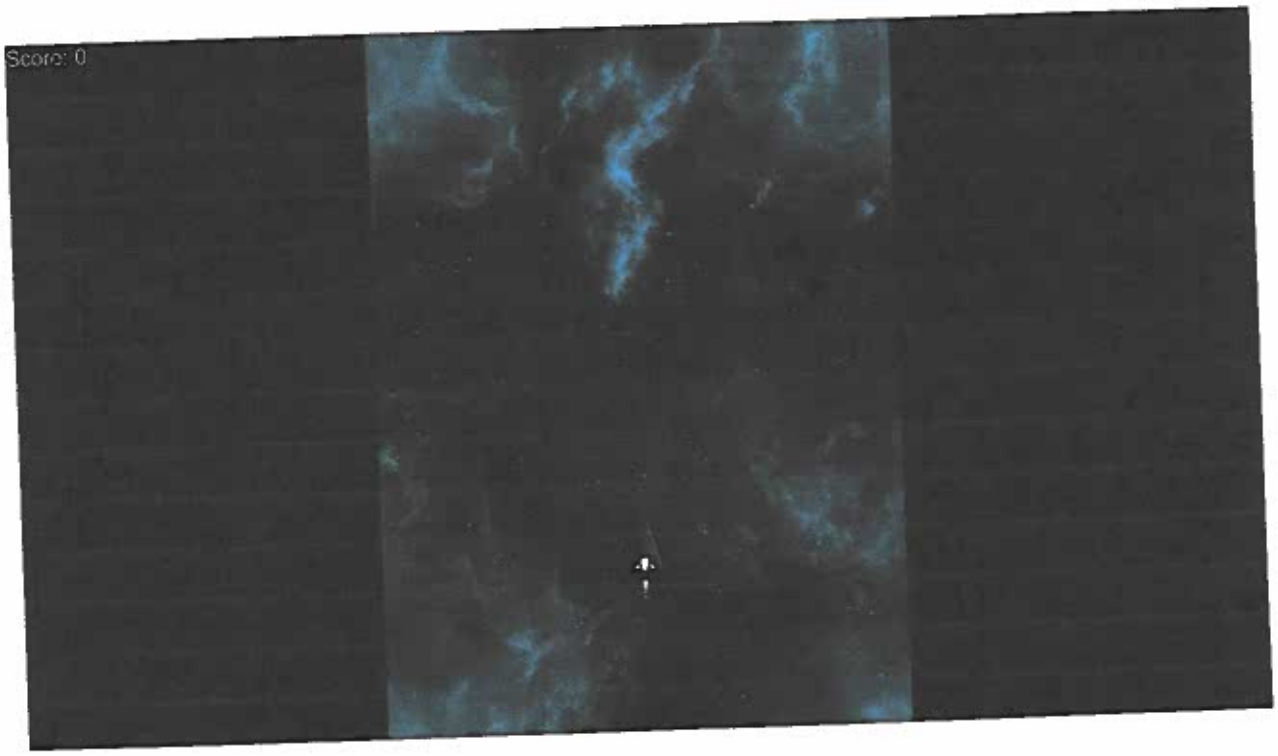
Background

For my senior project I decided to make a space shooter because video games have always held a special place in my life. I have always been fascinated by video games and my love and passion for them has only grown with them as technology gets better and better. But my favorite type of games is anything that has to do with space, specifically games like Galaga, Space invaders, and Defender. So when I got to pick what I wanted to do for my senior project I thought a video game was a great project to work on and what better game to make then a space shooter game. If anything I thought that this would challenge me as a programmer and give me some experience in a field that I didn't really know anything.

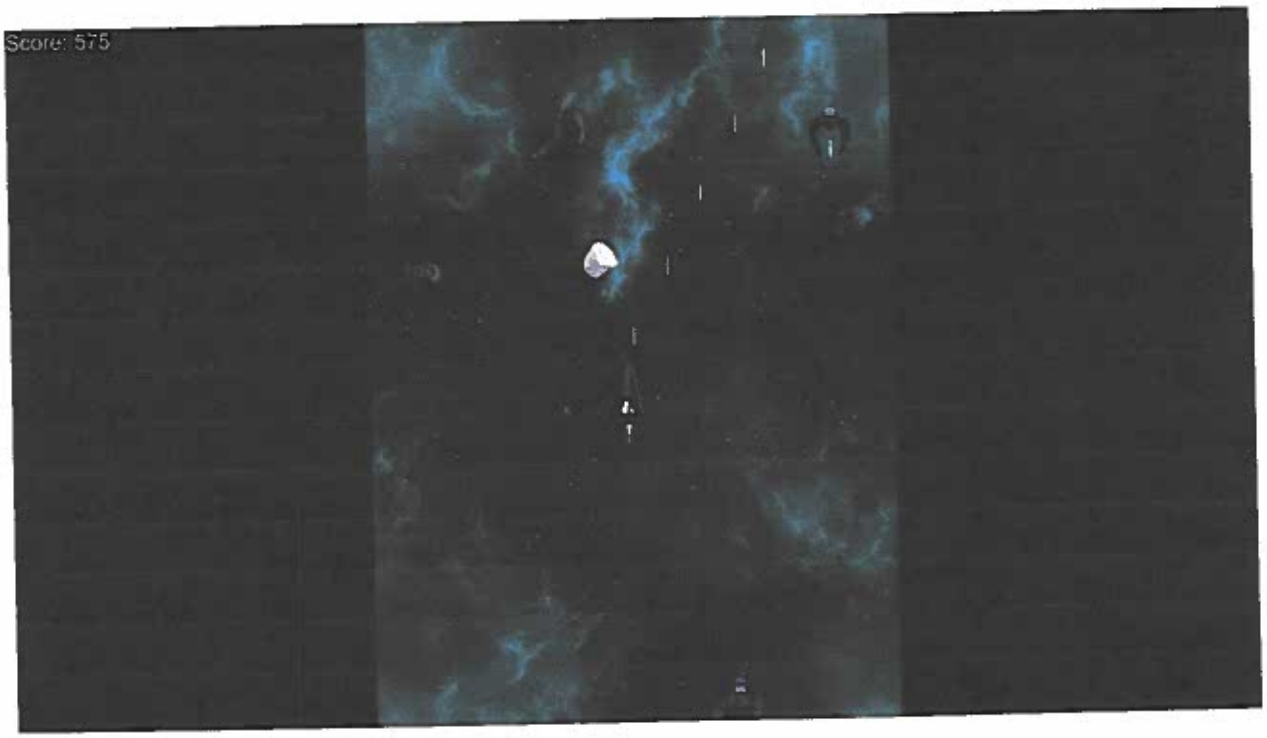
Hardware/Software

In order to make the greatness of my senior project happen I used Unity, Visual Studio, and C#. Unity is a game development software that uses a sophisticated UI to make programming for games a much more manageable task. It makes it easier to hookup different prefabs to code Scripts and then develop scenes that a user can play out in a game. I also used Unities Asset store to get all of my models and sound bites that I used in the project. I chose C# because Unities scripts are meant to be constructed using C# and defaults to Visual studio which automagically connects the code you construct to the physical game.

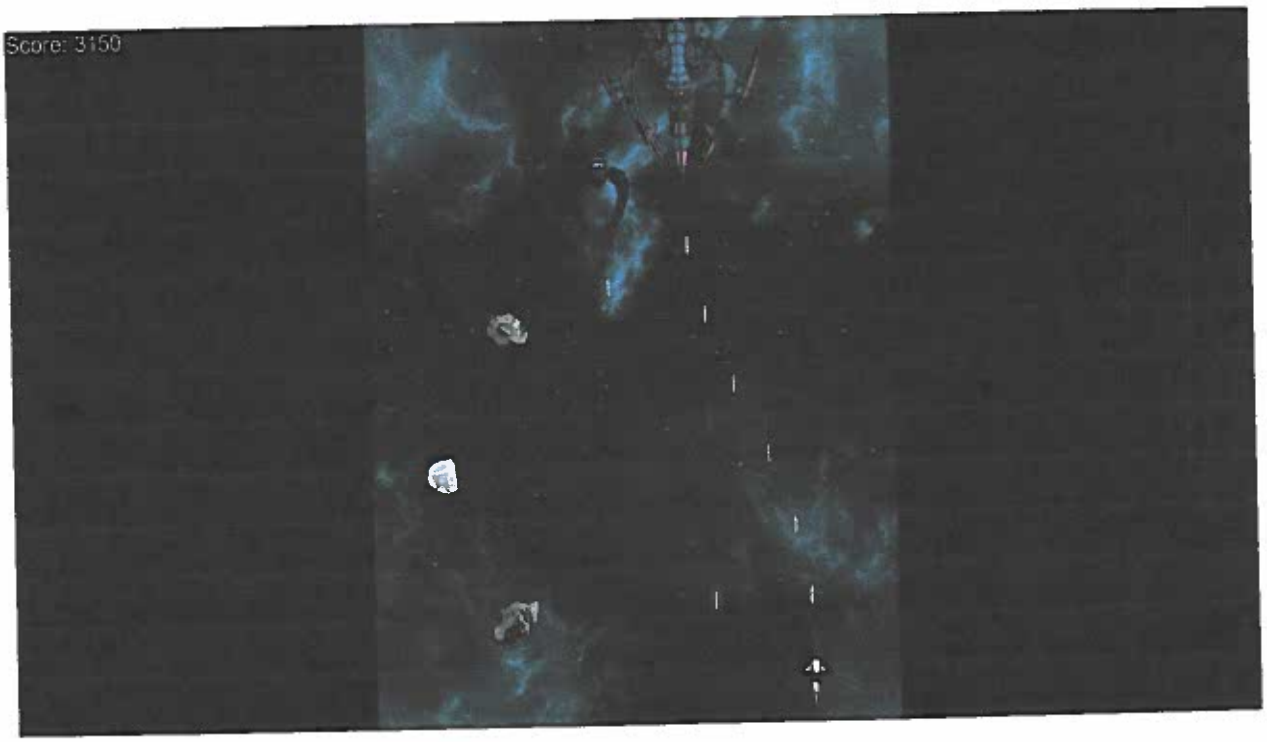
Final Project Screenshots/Explanation



- This is the initial start state of the game, as you can see the player starts out at the bottom of the screen and then the player waits for the enemy waves to start spawning and the initial score is 0 because the player hasn't done anything to earn a score.
- The background moves slowly to give the illusion that the player is moving through space and the player can move up, down, left, and right using the arrow keys and they can fire using the left CTRL button.



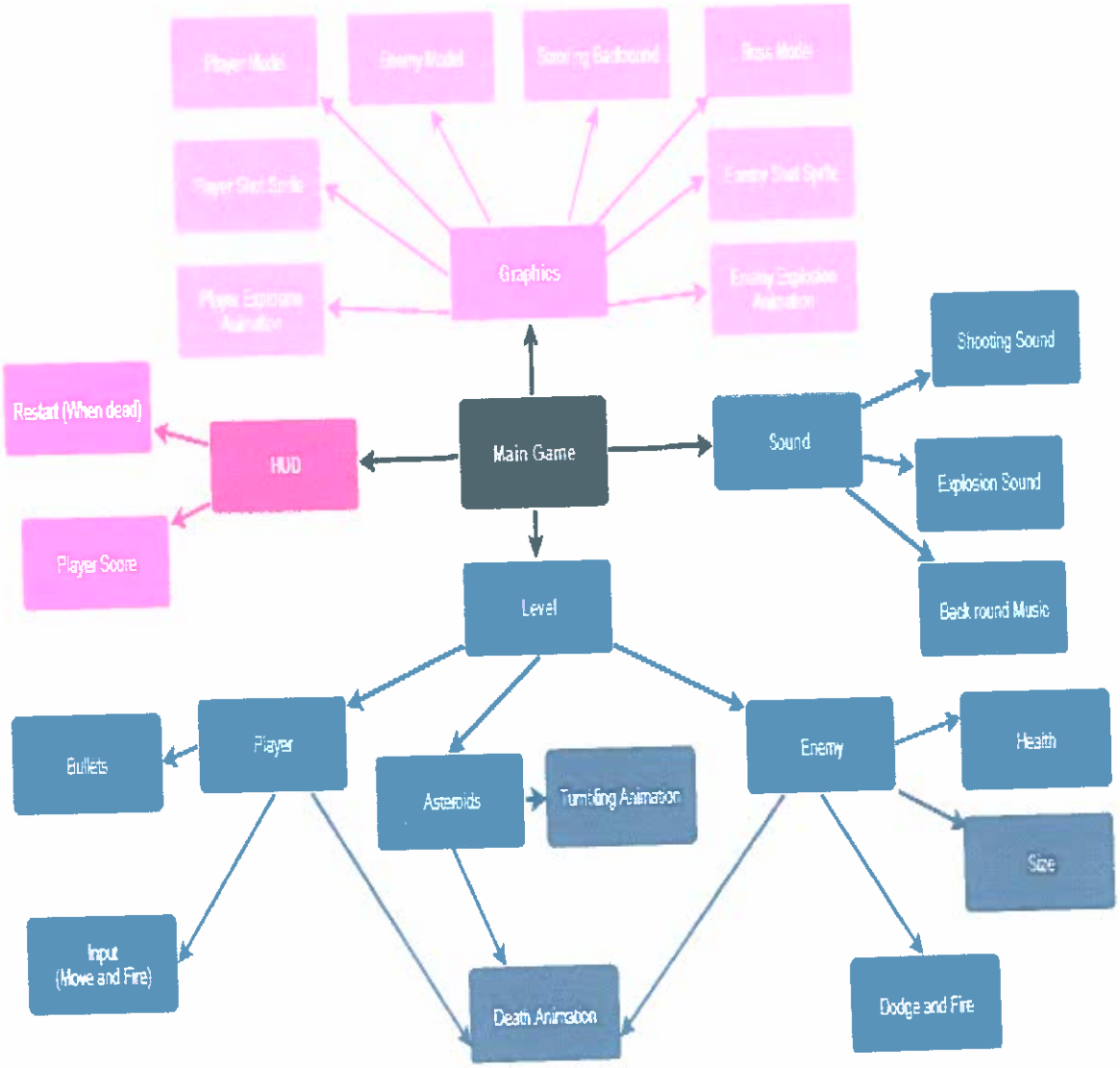
- After the initial start of the game then different enemies will spawn slightly off screen in front of the player and move down the screen and are deleted once they go off screen.
- As the enemies move down the screen they will fire shots at the enemy and dodge the players shots by dashing to the left or to the right depending on the position of the enemy.
- There are two kinds of enemies who dodge, there are ones who die in one hit and ones that take two hits and are slightly bigger.
- The asteroids that spawn all have different set speeds at which they come towards the player and tumble around to make it look like they are in space, the player can either shoot them or just dodge out of the way.



- Once the player has proven their skill by reaching the end of the wave of enemies then they are rewarded with a boss fight.
- The boss moves much slower down the screen then the other enemies and takes 50 shots in order to kill it.
- For right now the game just plays the next wave, but in my future development I would like to have the player go to a different level.

- When the player is hit by an enemy, bullet, or asteroid then the player blows up and the game over text appears on screen
- The enemies will stop spawning because the player isn't there to fight them
- Once the player has died they have the option to restart the game by pressing 'R'. Once this button is pressed the game will go back to its starting state, the players score is reset to 0, and they get another chance to face the wave of enemies.

Architecture Diagram



Test Plan

For my test plan I decided to do two different kinds of tests Playtests, and Unit Tests. I did Playtesting by just having 5 individuals at Charleston Southern University play my Game and let them offer feedback on what they thought I needed to improve and made and their information and what they offered are on the next couple of pages

The Unit tests were done by Unities test runner, which is a built in program that comes with the unity platform that allows a user to test different aspects of the game without messing up the rest of the project. It was a little tough because the easiest way to test a game is to just run it and see what needs to be fixed, but the Unit tests helped me test things without running

Test Results for Playtest

Male-To-Female Ratio: 3-2

Age-range 20 - 55

Participant #1

University: Charleston Southern University

Major: Political Science

Year: Senior

Suggestions for Game: Add more variety of enemies to the game

Implemented action: Added a new enemy that is bigger and takes two hits to kill

Participant #2

University: Charleston Southern University

Major: Biology

Year: Senior

Suggestions for Game: Add a way for the enemies to dodge the players shot

Implemented action: Added a function that allows enemies to evade player's shots

Participant #3

University: Charleston Southern University

Major: Computer Science

Year: Senior

Suggestions for Game: Fire rate is too slow, enemies too abundant for the fire rate of the player

Implemented action: Increased the fire rate and balanced enemy spawn rate to match the player's fire rate.

Participant #4

University: University of South Carolina

Major: Computer Science

Year: Junior

Suggestions for Game: Make the boss more challenging

Implemented action: Implemented a boss that has a high health pool and boss now has enemies that fight with it.

Participant #5

University: College of Charleston

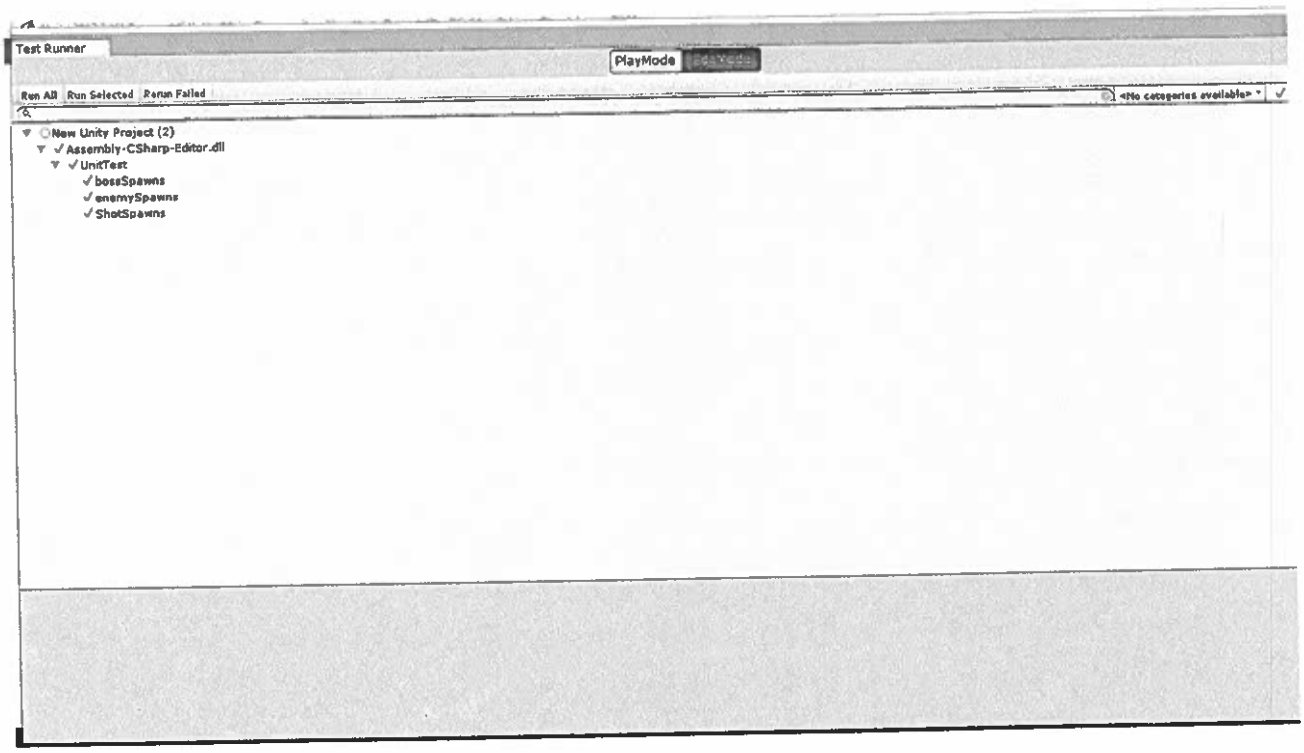
Major: Communication

Year: Sophomore

Suggestions for Game: Add different aspects to make the player feel like they are in space.

Implemented action: Added asteroids and starry background to enhance the feeling that the player is in space.

Unit Test Results



- The different kind of Unit tests I conducted were tests to make sure that the assets that the enemies used were the actual asset that they were supposed to use and to make sure that it actually spawned.
- The three tests I did was to make sure the enemies spawned, to make sure the boss spawned, and to make sure the bullets spawned when you fire.
- On the next page is the code for my Unit tests.

```
1  using UnityEngine;
2  using UnityEditor;
3  using UnityEngine.TestTools;
4  using NUnit.Framework;
5  using System.Collections;
6
7  public class UnitTest {
8
9
10     [UnityTest]
11     public IEnumerator ShotSpawns() {
12
13         var ShotPrefab = Resources.Load("_Complete-Game/Prefabs/Done_Bolt-Enemy")
             as GameObject;
14         var ShotSpawner = new GameObject().AddComponent<Weapons>();
15
16         var SpawnedShot = GameObject.FindWithTag("Enemy");
17         var SpawnedShotPrefab = PrefabUtility.GetPrefabParent(SpawnedShot);
18
19         ShotSpawner.Construct(ShotPrefab, 1);
20
21
22
23         Assert.AreEqual(ShotPrefab, SpawnedShotPrefab);
24         yield return null;
25     }
26
27     [UnityTest]
28     public IEnumerator bossSpawns()
29     {
30
31         var bossPrefab = Resources.Load("_Complete-Game/Prefabs/Done_Boss") as
             GameObject;
32         var bossSpawner = new GameObject().AddComponent<Game>();
33
34         var Spawnedboss = GameObject.FindWithTag("Enemy");
35         var SpawnedbossPrefab = PrefabUtility.GetPrefabParent(Spawnedboss);
36
37         bossSpawner.Construct1(bossPrefab);
38
39
40
41         Assert.AreEqual(bossPrefab, SpawnedbossPrefab);
42         yield return null;
43     }
44
45     [UnityTest]
46     public IEnumerator enemySpawns()
47     {
48
49         var enemyPrefab = Resources.Load("_Complete-Game/Prefabs/Done_Enemy Ship
             1") as GameObject;
```

```
50          var enemySpawner = new GameObject().AddComponent<Game>();
51
52          var Spawnedenemy = GameObject.FindWithTag("Enemy");
53          var SpawnedenemyPrefab = PrefabUtility.GetPrefabParent(Spawnedenemy);
54
55          enemySpawner.Construct2(enemyPrefab);
56
57
58
59          Assert.AreEqual(enemyPrefab, SpawnedenemyPrefab);
60          yield return null;
61      }
62  }
63
```

Challenges Overcome

- Lack of knowledge: At the start of this project I hadn't really used C# and I had never used Unity before, So getting to know both of those and making them mesh together was a bit of a challenge, but know I can add Unity and C# to the library of Coding languages and software that I know.
- Unity Crashes: There were multiple times when building this project that the Code I had written had some unseen bug and when I went to test to make sure what I implemented worked the entire Unity program would crash and I would have to reboot my project and try a different method. This really taught me to save frequently and often so that if it ever did crash I didn't lose a couple of hours of work.
- Trial and error: Like I said earlier the best way to test a Unity Game program is to Start playing it and see if it works and there were multiple times that I wasn't really sure what the problem was and I just had to keep Changing different segments of code to find the one that was the problem. This taught me to test often so that I could pinpoint where the code was failing.
- Senior Project: The project itself was very hard to keep up with because I had to do it while still focusing on my other classes. It was a very tough time, but now I am finally finished and I get to present my years of hard work.

Future Enhancements

- Enemy AI: I would like to add improved enemy AI and have the enemies loop back around if they aren't destroyed by the player to give an extra challenge.
- Different bosses: I would like to have multiple bosses that keeps a fresh look on different levels in the game.
- Scaling Difficulty: Right now the game is just one difficulty, but I would like to have the game get harder as you play it.
- Character Customization & upgrades: I would like to have a variety of ships for the player to choose from and give the player the ability to collect power-ups to make the game more manageable as it gets harder.
- Different levels and backgrounds: I would also like to have different levels and backgrounds to make the player feel like they are traveling throughout space to defeat the enemies.

3-D Space Shooter

By: Jonah King

Advisor: Dr. Paul West

Major: Bachelor of Science in Computer Science

Why Did I decide to make a 3D Space Shooter?

- Nostalgia- when I was little I used to love to play games like Galaga, Space Invaders, and Defender
- Experience- Game developing is something that I would like to learn so I decided to try my hand at it.
- Fun- I thought this would be a fun project to work on and since the Senior project feels like a daunting task, it was easier when I got to work on something I enjoyed.

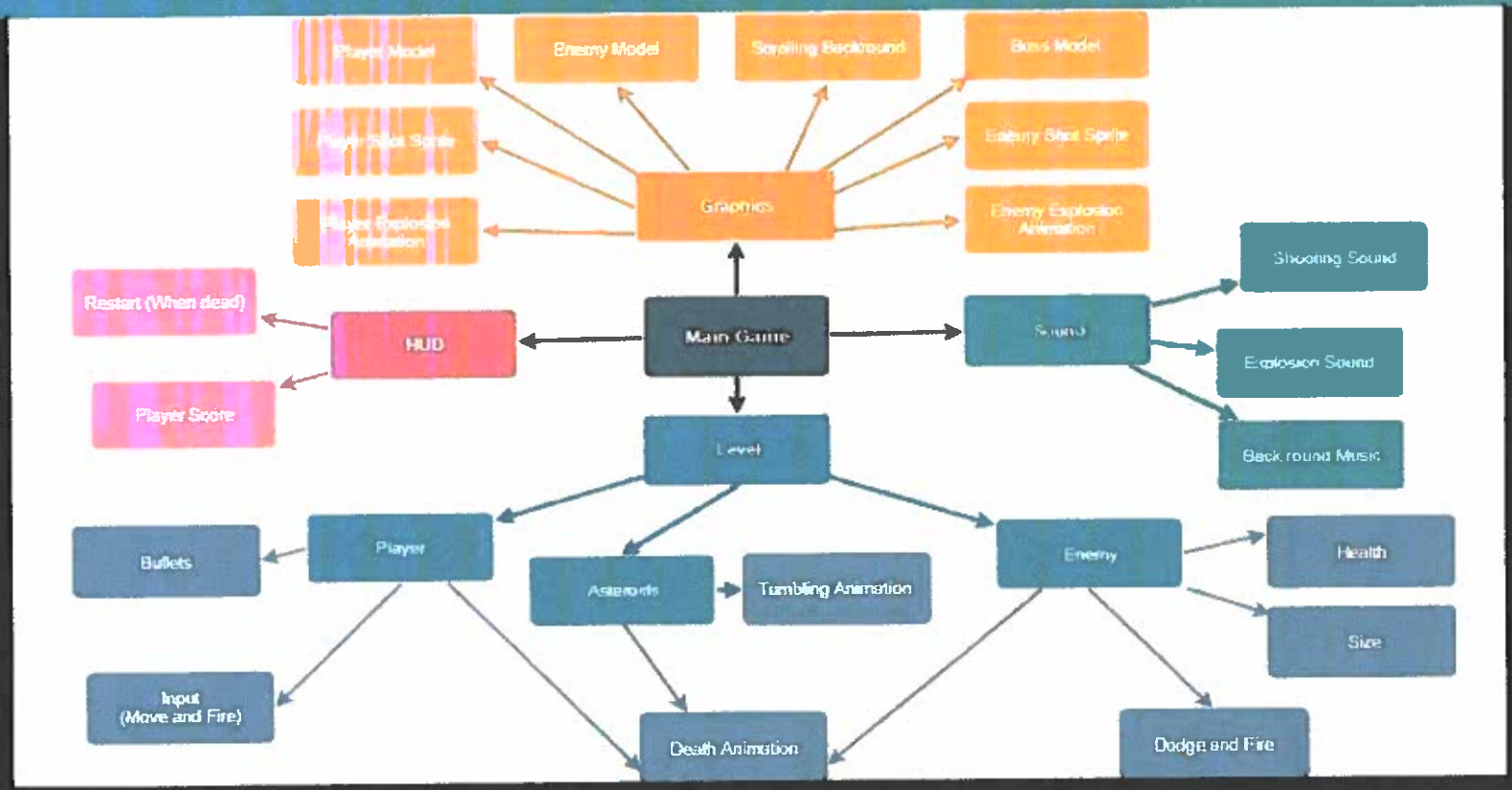
What would be required

- Game Engine- For this I decided to go with the Unity engine as it is specifically made for game design and has many built in functions that would make life easier
- Programming Language and Editor- I used C# and Visual Studio because that is what Unity defaults to and what was the easiest method to make my project happen.
- Artwork- I got my 3D artwork from Unity's Asset store all free of charge.

Lets See It in Action

C:\Users\Jonah\Desktop\Untitled 4.avi

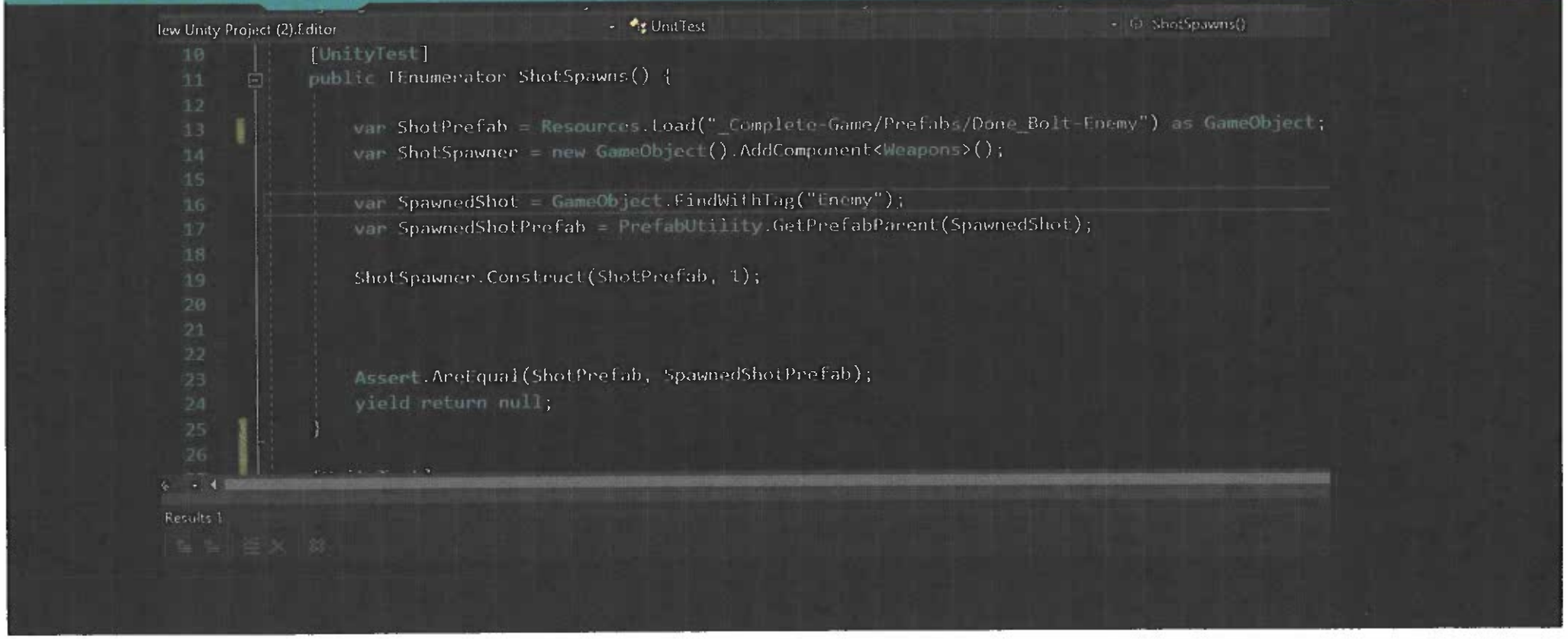
Architecture Diagram



How Did I Test it?

- I got five participants to play my game and give me feedback on what they thought needed to be changed.
- After I received feedback from a person I would evaluate what they said and make changes to the game.

Unit Testing



Future Enhancements

- Improved Enemy AI
- Different Bosses
- Character customization and upgrades
- Scaling difficulty
- Different levels/backgrounds

The End

- Any Questions, Concerns, Comments, Or Screams?

```
1  using UnityEngine;
2  using UnityEngine.SceneManagement;
3  using System.Collections;
4  using UnityEngine.UI;
5
6  public class Game : MonoBehaviour
7  {
8      public GameObject[] Enemies;
9      public GameObject strongEnemy;
10     public GameObject bigBoss;
11
12     public Vector3 spawnBoss;
13     public Vector3 spawnValues;
14     public int EnemyCount;
15     public float enemySpawn;
16     public int gameStart;
17     public int waveStart;
18    // Future implementation... public float dropRate;
19
20     public Text scoreText;
21     public Text StartOverText;
22     public Text gameOverText;
23
24     private bool gameOver;
25     private bool StartOver;
26     private int score;
27
28
29
30     // For Unit testing
31     public void Construct1(GameObject copyBoss)
32     {
33         bigBoss = copyBoss;
34     }
35
36     public void Construct2(GameObject copyenemy)
37     {
38         strongEnemy = copyenemy;
39     }
40     /*
41      * initializes all values to their proper Starting state and calls the
          function
42      * SpawnWaves which Starts the game
43      */
44     void Start()
45     {
46         gameOver = false;
47         StartOver = false;
48         StartOverText.text = "";
49         gameOverText.text = "";
50         score = 0;
51         UpdateScore();
```

```
52          StartCoroutine(SpawnWaves());
53      }
54
55      /*
56       * This Checks to see if the players ship has exploded and sets up for the
           player to be
57       * able to reStart the game by pressing the 'R' key and reloads the game
58       */
59
60      void Update()
61      {
62          if (StartOver)
63          {
64              if (Input.GetKeyDown(KeyCode.R))
65              {
66                  SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);
67              }
68          }
69      }
70      // Function that spawns waves of enemies
71      IEnumerator SpawnWaves()
72      {
73          //Wait time befor the game Starts
74          yield return new WaitForSeconds(gameStart);
75          while (true)
76          {
77              //sets up a loop that will happen for as many enemies as you give the
                   global variable
78              for (int i = 0; i < EnemyCount; i++)
79              {
80                  /*
81                   * selects a random object in the array of enemies and puts them
                       in a
82                   * random position off screen in front of the player, then
                       instantiates
83                   * the enemy prefab
84                   */
85                  GameObject hazard = Enemies[Random.Range(0, Enemies.Length)];
86                  Vector3 spawnPosition = new Vector3(Random.Range(-spawnValues.x,
                      spawnValues.x), spawnValues.y, spawnValues.z);
87                  Quaternion spawnRotation = Quaternion.identity;
88                  Instantiate(hazard, spawnPosition, spawnRotation);
89                  //future Enhancement...
90                  /*if (Random.Range(0, 1) <= dropRate)
91                  {
92                      var pickupdrop = Instantiate(drop, hazard.transform.position,
                          Quaternion.identity);
93                  }*/
94
95                  /*
96                   * waits until the end of the wave of enemies then spawns
97                   * the boss in the same way it spawns enemies.
```

```
98                  */
99                 bool bossTime = false;
100                if(i == 98){ bossTime = true; }
101                if (bossTime == true)
102                {
103                        GameObject Boss = bigBoss;
104                        Vector3 spawnPositionn = new Vector3(Random.Range(-
                       spawnBoss.x, spawnBoss.x), spawnBoss.y, spawnBoss.z);
105                        Quaternion spawnRotationn = Quaternion.identity;
106                        Instantiate(Boss, spawnPositionn, spawnRotationn);
107                }
108                /*
109                 * If at any point in the Game the player dies this will set the
                    StartOverText
110                 * to a Text the player can see and allows the player to break
                    the loop
111                 * and the player can reStart.
112                 */
113                yield return new WaitForSeconds(enemySpawn);
114                if (gameOver)
115                {
116                    StartOverText.text = "Press 'R' for StartOver";
117                    StartOver = true;
118                    break;
119                }
120            }
121            //Waits to Start the next wave
122            yield return new WaitForSeconds(waveStart);
123
124
125        }
126    }
127
128
129    // Adds the Score from enemies the player destroins and updates the score
130    public void AddScore(int newScoreValue)
131    {
132        score += newScoreValue;
133        UpdateScore();
134    }
135
136    // Shows the player the updated score
137    void UpdateScore()
138    {
139        scoreText.text = "Score: " + score;
140    }
141
142    //Tells the player when they have lost the game
143    public void GameOver()
144    {
145        gameOverText.text = "Game Over!";
146        gameOver = true;
```

```
147     }
148 }
```

```
1  using UnityEngine;
2  using System.Collections;
3
4  [System.Serializable]
5  public class Boundary
6  {
7      public float xMin, xMax, zMin, zMax;
8  }
9
10 public class PlayerControls : MonoBehaviour
11 {
12     public int playerSpeed;
13     public float tilting;
14     public Boundary playerBoundary;
15
16     public GameObject shot;
17     public Transform shotSpawn;
18     public float RateofFire;
19
20     private float nextShot;
21
22     /*
23      * This function gets the the input of a fire button and fires as many
24      * times as is aloted by the fire rate, spawns the shot at the players current
           location,
25      * and plays the corrisponding audio.
26      */
27     void Update ()
28     {
29         if (Input.GetButton("Fire1") && Time.time > nextShot)
30         {
31             nextShot = Time.time + RateofFire;
32             Instantiate(shot, shotSpawn.position, shotSpawn.rotation);
33             GetComponent<AudioSource>().Play ();
34         }
35     }
36
37     /*
38      * This function gets the controls from the user and moves the Players ship
39      * According to what the player input.
40      */
41     void FixedUpdate ()
42     {
43         float HorizontalMovement = Input.GetAxis ("Horizontal");
44         float VerticalMovement = Input.GetAxis ("Vertical");
45
46         Vector3 movingPlayer = new Vector3 (HorizontalMovement, 0.0f,
             VerticalMovement);
47         GetComponent<Rigidbody>().velocity = movingPlayer * playerSpeed;
48
49         GetComponent<Rigidbody>().position = new Vector3
50         (
```

```
51              Mathf.Clamp (GetComponent<Rigidbody>().position.x,
                  playerBoundary.xMin, playerBoundary.xMax),
52              0.0f,
53              Mathf.Clamp (GetComponent<Rigidbody>().position.z,
                  playerBoundary.zMin, playerBoundary.zMax)
54          );
55
56          GetComponent<Rigidbody>().rotation = Quaternion.Euler (0.0f, 0.0f,
              GetComponent<Rigidbody>().velocity.x * -tilting);
57      }
58  }
59
```

```
1  using UnityEngine;
2  using System.Collections;
3
4  public class Weapons : MonoBehaviour
5  {
6      public GameObject Bullet;
7      public Transform spawnShot;
8      public int RateofFire;
9      public float Pause;
10
11     // For Unit testing
12     public void Construct(GameObject bullets, float pause)
13     {
14         Bullet = bullets;
15         Pause = pause;
16     }
17
18     void Start ()
19         //Repeats the action after Pause value of seconds has passed
20     {
21         InvokeRepeating ("Shoot", Pause, RateofFire);
22     }
23
24     void Shoot ()
25     {
26         //Spawns the bullet in the position that the Enemy is currently located
27         Instantiate(Bullet, spawnShot.position, spawnShot.rotation);
28         //Plays the audio that is inserted every time code is called
29         GetComponent<AudioSource>().Play();
30     }
31 }
32
```

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class Tumbler : MonoBehaviour {
6
7  public int movement;
8
9  // gets the movement speed of the object and assigns a rate at which the object
     tumbles around
10     void Start()
11     {
12     GetComponent<Rigidbody>().angularVelocity = Random.insideUnitSphere *
         movement;
13     }
14 }
15
```

```
1  using UnityEngine;
2  using System.Collections;
3
4  public class ScrollingBackground : MonoBehaviour
5  {
6      public int movementSpeed;
7      public int interval;
8
9      private Vector3 beginningPos;
10     //gets the Starting position of the scrolling Backround
11     void Start ()
12     {
13         beginningPos = transform.position;
14     }
15     /*
16      * constantly updates the position of the Background and dtores the new value, ➪
17      * so it always appears to be moving.
18      */
19     void Update ()
20     {
21         float update = Mathf.Repeat(Time.time * movementSpeed, interval);
22         transform.position = beginningPos + Vector3.forward * update;
23     }
24 }
```

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5
6  public class KillByBoundary : MonoBehaviour
7  {
8      //This makes sure that after a object has gone outside of its boundary it will ↵
          be destroyed
9      void OnTriggerExit(Collider other)
10     {
11         Destroy(other.gameObject);
12     }
13 }
14
```

```
1  using UnityEngine;
2  using System.Collections;
3
4  public class HealthofEnemy : MonoBehaviour
5  {
6      public GameObject explosion;
7      public GameObject playerExplosion;
8      public int health;
9      public int scoreValue;
10     private Game gameController;
11
12
13     /*
14      * Debugging Code that makes sure that the Game can Start and gets components
15      * from the Game script.
16      */
17
18     void Start()
19     {
20         GameObject gameControllerObject = GameObject.FindGameObjectWithTag
             ("GameController");
21         if (gameControllerObject != null)
22         {
23             gameController = gameControllerObject.GetComponent<Game>();
24         }
25         if (gameController == null)
26         {
27             Debug.Log("Cannot find 'GameController' script");
28         }
29     }
30
31     /*
32      * This function hapens whenever any sort of contact is made and makes sure
          that
33      * the appropriate action is taken based on what collides with each other
34      */
35     void OnTriggerEnter(Collider other)
36     {
37         while (health > 0)
38         {
39             if (other.tag == "Boundary" || other.tag == "Enemy")
40             {
41                 return;
42             }
43
44             if (other.tag == "Projectile")
45             {
46                 health--;
47                 Destroy(other.gameObject);
48                 return;
49             }
50             if (other.tag == "Player")
```

```
51              {
52                  Instantiate(playerExplosion, other.transform.position,
                      other.transform.rotation);
53                  gameController.GameOver();
54                  Destroy(other.gameObject);
55                  return;
56
57              }
58          }
59
60
61          if (health == 0)
62          {
63            Instantiate(explosion, transform.position, transform.rotation);
64              gameController.AddScore(scoreValue);
65              Destroy(other.gameObject);
66              Destroy(gameObject);
67          }
68
69      }
70  }
71
72
```

```
1  using UnityEngine;
2  using System.Collections;
3
4  public class EvasiveManeuver : MonoBehaviour
5  {
6      public Boundary boundary;
7      public float tilting;
8      public float Dashing;
9      public float Stabalizing;
10     public Vector2 startManeuver;
11     public Vector2 timeManeuver;
12     public Vector2 waitManeuver;
13
14     private float MovementSpeed;
15     private float enemyManeuver;
16
17     /*
18      * Gets the movementspeed of the enemy and starts the Evade function.
19      */
20     void Start ()
21     {
22         MovementSpeed = GetComponent<Rigidbody>().velocity.z;
23         StartCoroutine(Evade());
24     }
25
26     /*
27      * This function gets the different values to use for the evading maneuver
28      */
29     IEnumerator Evade ()
30     {
31         yield return new WaitForSeconds (Random.Range (startManeuver.x,
             startManeuver.y));
32         while (true)
33         {
34             enemyManeuver = Random.Range (1, Dashing) * -Mathf.Sign
               (transform.position.x);
35             yield return new WaitForSeconds (Random.Range (timeManeuver.x,
               timeManeuver.y));
36             enemyManeuver = 0;
37             yield return new WaitForSeconds (Random.Range (waitManeuver.x,
               waitManeuver.y));
38         }
39     }
40
41     /*
42      * This Function executes the Evade function and makes the enemy carry out the
          function
43      */
44     void FixedUpdate ()
45     {
46         float newManeuver = Mathf.MoveTowards (GetComponent<Rigidbody>
             ().velocity.x, enemyManeuver, Stabalizing * Time.deltaTime);
```

```
47          GetComponent<Rigidbody>().velocity = new Vector3 (newManeuver, 0.0f,
              MovementSpeed);
48          GetComponent<Rigidbody>().position = new Vector3
49          (
50              Mathf.Clamp(GetComponent<Rigidbody>().position.x, boundary.xMin,
                  boundary.xMax),
51              0.0f,
52              Mathf.Clamp(GetComponent<Rigidbody>().position.z, boundary.zMin,
                  boundary.zMax)
53          );
54
55          GetComponent<Rigidbody>().rotation = Quaternion.Euler (0, 0,
              GetComponent<Rigidbody>().velocity.x * -tilting);
56      }
57  }
58
```

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class ContactDestroyer : MonoBehaviour {
6
7      public GameObject enemyExplosion;
8      public GameObject playerExplosion;
9      public int scoreValue;
10     private Game gameController;
11
12     /*
13      * Makes sure it is possible to reach the Game script and gets components from
           the script
14      */
15     void Start()
16     {
17         GameObject gameControllerObject = GameObject.FindGameObjectWithTag
             ("GameController");
18         if (gameControllerObject != null)
19         {
20             gameController = gameControllerObject.GetComponent<Game>();
21         }
22         if (gameController == null)
23         {
24             Debug.Log("Cannot find 'GameController' script");
25         }
26     }
27
28     /*
29      * Makes sure that on contact the trigger causes different objects to be
           destroyed
30      * when they come in contact with each other and makes sure that their
           existance is
31      * Destroyed
32      */
33     void OnTriggerEnter(Collider other)
34     {
35         if (other.tag == "Boundary" || other.tag == "Enemy")
36         {
37             return;
38         }
39
40         if (enemyExplosion != null)
41         {
42             Instantiate(enemyExplosion, transform.position, transform.rotation);
43         }
44
45         if (other.tag == "Player")
46         {
47             Instantiate(playerExplosion, other.transform.position,
                 other.transform.rotation);
```

```
48              gameController.GameOver();
49          }
50
51          gameController.AddScore(scoreValue);
52          Destroy(other.gameObject);
53          Destroy(gameObject);
54      }
55  }
56
```

```
1  using UnityEngine;
2  using System.Collections;
3
4  public class AutoMover : MonoBehaviour
5  {
6      public int movementSpeed;
7
8
9      /*(This function is used to select an automatic moving speed that can be as
         fast or as
10      * slow as needed and is used for enemies and bullets
11      */
12     void Start ()
13     {
14         GetComponent<Rigidbody>().velocity = transform.forward * movementSpeed;
15     }
16 }
17
```